Software Engineering Project · Coding Phase

AI-Powered Co-Working Space Asset Manager

Team coding plan: who builds what, and how we put it together

Misbah Ul Hasan
CS-24130 · AI (server side) + Dashboard

Hooria Ahmed
CS-24106 · AI (screens) + Notifications + History

Syed Azlaan Aqib
CS-24144 · Assets + Search + Maintenance

M. Hamza Siddiqui
CS-24139 · Requests + Approval + Borrowing + Users

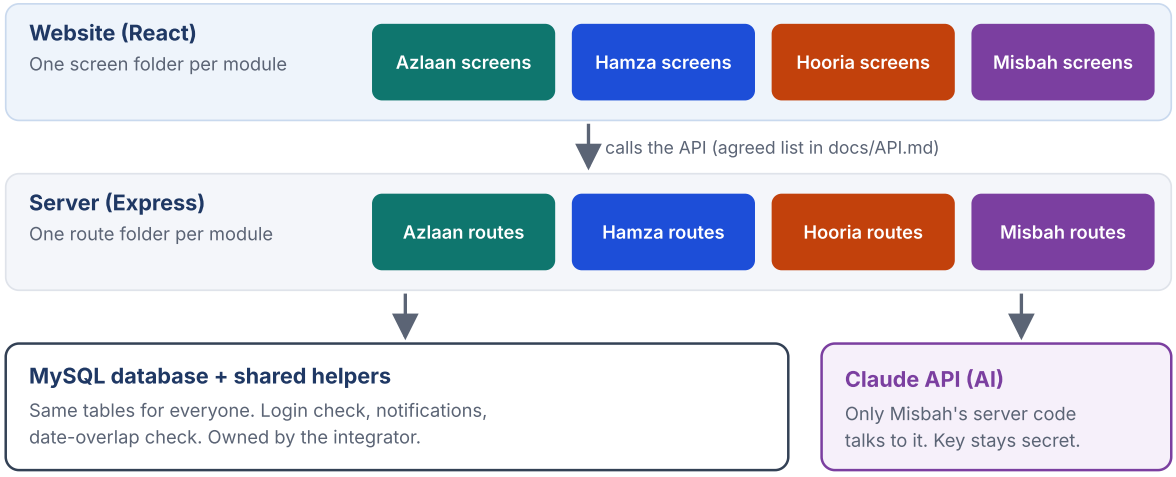
Integrator: Shahan Waheed
Combines everyone's modules, tests the full app and puts it live

Built with: React (website) · Node.js + Express (server) · MySQL (database) · JWT (login) · Claude API (AI features)
Requirements: every task below has an SRS ID (for example FR-RQ-02) so you can look it up in the SRS.

1. The big idea

> We already have a **starter app that runs**. It has login, the database with sample assets, and the menu. Every module has an **empty slot** waiting for its owner. Each person fills **only their own slots**, so when we combine the work, nothing clashes.

How the pieces fit



What is already done for you

Ready in the starter app	What it means for you
Login and register	You can log in as admin or member and test your screens straight away.
Database with sample data	10 assets, 6 categories, an admin and a member are created by one command.
Menu and every page registered	Your page already appears in the menu as a placeholder card with your name on it.
Every endpoint registered	Your endpoints already exist and answer "not implemented yet". You replace that with real code.
Shared helpers	Database connection, login check, admin check, status names, notifications, date overlap check, AI client.

Test accounts

Role	Email	Password
Admin	`admin@cowork.local`	`Admin@123`
Member	`member@cowork.local`	`Member@123`

2. Who does what

Each person builds **both halves** of their module: the server part (routes) and the website part (screens). The one exception is the AI: **Misbah builds the server side and Hooria builds the screens**, so they work as a pair.

Member	Module	SRS IDs	Screens
Misbah	**AI Asset Recommendation** (server)	FR-RC-01 to 07	(Hooria)
	AI Asset Assistant (server)	FR-AI-01 to 09	(Hooria)
	Dashboard and Reports	FR-RP-01 to 05	Admin Dashboard, Reports
Hooria	**AI Asset Recommendation** (screen)	FR-RC-01, 04, 05, 06	AI Recommendation
	AI Asset Assistant (screen)	FR-AI-01, 07	AI Assistant chat
	Notifications	FR-NT-01 to 03	Bell icon, Notifications
	Usage History	FR-UH-01 to 03	My History, Asset History
Azlaan	Asset Management + Categories	FR-AM-01 to 07	Manage Assets, Categories
	Asset Search and Details	FR-AS-01 to 05	Asset Catalogue, Asset Details
	Maintenance	FR-MT-01 to 05	Maintenance, Report a Problem
Hamza	Asset Request	FR-RQ-01 to 06	Request form, My Requests
	Request Approval	FR-AP-01 to 06	Approvals
	Borrowing and Returns	FR-BR-01 to 06	Issue / Return
	Manage Users	FR-AU-08	Users
Integrator	Starter app, login, database, combining, going live	FR-AU-01 to 07	Login, Register, menu

3. Rules and connections

The 5 golden rules

1. **Only change your own folders.** Your code goes in `server/src/modules/<your module>/` and `client/src/modules/<your module>/`. Shared files belong to the integrator; ask in the group if you need one changed.

2. **Stick to the agreed API.** `docs/API.md` lists every endpoint with what it takes and returns. Other people's screens use your endpoints exactly as written. If you change one, tell them first.

3. **Use the shared helpers, don't copy them.** Database, login check, status names, notifications and the date overlap check already exist. Use them.

4. **Errors look the same everywhere:** `{ "error": "message" }` with a proper status code, shown on screen with `<ErrorBox>`.

5. **Never change the database tables yourself.** Need a new column? Ask the integrator so everyone gets it.

Where our modules connect

These are the only places where one person's work touches another's. Agree on them early.

When this happens...	...it connects to	How
Azlaan member clicks "Request" on an asset	Hamza request form	Opens `/request/<asset id>`
Hooria member clicks "Request" on a recommendation	Hamza request form	Opens `/request/<id>?start=...&days=...`
Hamza approves, rejects or marks overdue	Hooria notifications	Calls `notify(user, type, message)`
Azlaan member reports a problem	Hooria notifications	Calls `notifyAdmins(...)`
Misbah AI picks free assets	Hamza overlap check	Both use `availability.js`
Misbah dashboard shows overdue items	Hamza borrow records	Reads the `borrow_records` table
Azlaan admin clicks "View history"	Hooria asset history	Opens `/admin/history/<asset id>`
Hooria AI screens	Misbah AI server	`POST /api/ai/chat` and `POST /api/ai/recommend`

Misbah Ul Hasan

AI server side (both AI features) + Dashboard and Reports · partner: Hooria

Your folders: `server/src/modules/ai/`, `server/src/services/llm.js`, `server/src/modules/reports/`, `client/src/modules/reports/`

A. AI Recommendation (FR-RC)

The member describes what they need ("laptop for video editing, 3 days") and gets the 5 best **available** assets, each with a reason.

- [ ] Check the form: need, purpose, start date (today or later), 1 to 30 days
- [ ] Get only free assets with `getAvailableAssets(start, end)` (FR-RC-02)
- [ ] Send the need + that list to Claude; ask for the top 5 with one-line reasons (FR-RC-03, 04)
- [ ] Ignore any asset the AI names that was not in the list (so it can never invent one)
- [ ] No AI key or AI fails: rank by keyword matches instead (FR-RC-07)
- [ ] Nothing free: say so and suggest the nearest free date (FR-RC-06)

B. AI Assistant chat (FR-AI)

The member asks questions in plain words. The AI does **not** read the database itself: it asks your code to run a lookup, you run the SQL and give it the result.

- [ ] Give the AI 4 read-only lookups: search assets, check availability, my requests, list categories (FR-AI-02, 06)
- [ ] Run each lookup it asks for and send the results back until it answers (FR-AI-03)
- [ ] Tell it: answer only from the lookups, never invent assets (FR-AI-04)
- [ ] "My requests" returns only the logged-in user's rows (FR-AI-05)
- [ ] AI down: friendly "assistant unavailable" message (FR-AI-07)
- [ ] Save every question and answer; max 50 AI uses per person per day (FR-AI-08, 09)

C. Dashboard and Reports (FR-RP)

- [ ] Dashboard: number of assets in each status, overdue list, pending requests (FR-RP-01, 02)
- [ ] Charts: most-used assets and use per category for a date range (FR-RP-03)
- [ ] Maintenance summary: open repairs and total cost (FR-RP-04)
- [ ] Export a report as CSV (PDF optional) (FR-RP-05)

> **You're done when:** "video editing for 3 days" lists only free assets; asking about a "Sony drone" gets "it doesn't exist"; with no AI key, recommendations still appear.

The step-by-step code example for the AI loop is in `docs/modules/misbah.md`.

Hooria Ahmed

AI screens (both AI features) + Notifications + Usage History · partner: Misbah

Your folders: `client/src/modules/ai/`, `notifications/` and `history/` (server and client), `server/src/services/notify.js`

Tip: you don't have to wait for Misbah. Start with fake data shaped exactly like `docs/API.md` says, then switch to the real call when her part is ready.

A. AI Recommendation screen (FR-RC)

- [ ] Form: what you need, purpose, start date, number of days (FR-RC-01)
- [ ] Loading message while the AI thinks (it can take up to 10 seconds)
- [ ] Show up to 5 results in order: picture, name, location, and the AI's reason (FR-RC-04)
- [ ] "Request" button on each result opens Hamza's form with the dates already filled in (FR-RC-05)
- [ ] No results: show the message and the nearest free date (FR-RC-06)

B. AI Assistant chat screen (FR-AI)

- [ ] Chat bubbles and a text box with a 500-character counter (FR-AI-01)
- [ ] "Thinking..." while waiting; errors shown inside the chat, never a blank page (FR-AI-07)
- [ ] "12 of 50 AI requests used today" counter
- [ ] Example question buttons: "Is any projector free tomorrow?"

C. Notifications (FR-NT)

- [ ] List my notifications, newest first; mark one or all as read (FR-NT-01, 03)
- [ ] Bell icon in the top bar with the unread count, refreshed every minute
- [ ] Reminder one day before an item is due (FR-NT-01)
- [ ] Keep the `notify()` helper's name the same; Hamza and Azlaan use it

D. Usage History (FR-UH)

- [ ] Member: my past requests and borrowings (FR-UH-02)
- [ ] Admin: everything that ever happened to one asset (borrowings and repairs) (FR-UH-01)
- [ ] Read only: no edit or delete buttons anywhere (FR-UH-03)

You're done when: clicking "Request" on a recommendation opens a pre-filled form; approving a request makes the member's bell count go up.

Syed Azlaan Aqib

Assets, Categories, Search and Details, Maintenance

Your folders: `assets/`, `categories/`, `maintenance/` (server and client)

Do this first: everyone's screens show assets, so build the asset list and asset details endpoints first and share them early.

A. Categories (FR-AM-07)

- [ ] Add, rename and delete categories (can't delete one that still has assets)

B. Manage Assets (FR-AM)

- [ ] Admin adds an asset: tag, name, category, location, condition, status, description, picture (JPG/PNG, max 2 MB) (FR-AM-01)
- [ ] Two assets can't share a tag; show "Asset tag already exists" (FR-AM-02)
- [ ] Condition and status only from the fixed lists (FR-AM-03, 04)
- [ ] Edit an asset; can't delete one that is Reserved or Borrowed (FR-AM-05, 06)

C. Search and Asset Details (FR-AS)

- [ ] Search box (name, tag, description) + filters for category, location, status, condition (FR-AS-01, 02)
- [ ] 20 results per page with picture and status badge (FR-AS-03)
- [ ] Details page shows everything plus the dates it's already booked (FR-AS-04)
- [ ] "Request" button only for members and only when the asset is Available (FR-RQ-01)
- [ ] Members never see Retired assets (FR-AS-05)

D. Maintenance (FR-MT)

- [ ] Admin opens a repair: asset becomes Under Maintenance (FR-MT-01, 02)
- [ ] Add notes and cost (FR-MT-03)
- [ ] Close it as Repaired (asset Available again) or Retired (FR-MT-04)
- [ ] Member reports a problem with something they borrowed; admins get notified (FR-MT-05)

You're done when: searching "laptop" on Floor 2 shows only the MacBook and the ThinkPad; deleting a borrowed asset is refused; closing a repair makes the asset Available.

M. Hamza Siddiqui

Requests, Approval, Borrowing and Returns, Manage Users

Your folders: `requests/`, `approvals/`, `borrowing/`, `users/` (server and client), `server/src/services/availability.js`

You build the main journey of the app. Follow the two state diagrams in the SRS (asset and request) exactly:

Member requests → Pending → Admin approves → Asset Reserved → Admin issues → Asset Borrowed → Returned → Available (or Under Maintenance if damaged)

A. Asset Request (FR-RQ)

- [ ] Form: purpose, start date (not in the past), 1 to 30 days; saved as Pending (FR-RQ-02, 03, 04)
- [ ] Pre-fill the dates when opened from an AI recommendation (`?start=...&days=...`)
- [ ] My Requests list with status; cancel while Pending (FR-RQ-05, 06)
- [ ] Tell the admins a new request arrived

B. Approval (FR-AP)

- [ ] List pending requests (FR-AP-01)
- [ ] Approve: refuse if the dates overlap another booking; asset becomes Reserved (FR-AP-02, 03)
- [ ] Reject with a reason (FR-AP-04); notify the member either way (FR-AP-05)
- [ ] Cancel an approved request that wasn't collected (FR-AP-06)

C. Borrowing and Returns (FR-BR)

- [ ] Issue: asset becomes Borrowed, due date recorded (FR-BR-01)
- [ ] Return with condition; Damaged sends it to maintenance; request Completed (FR-BR-02, 03, 06)
- [ ] Hourly check marks late items Overdue and notifies the borrower (FR-BR-04, 05)

D. Manage Users (FR-AU-08)

- [ ] Admin sees all users and can deactivate / reactivate them (never themselves)

You're done when: two members ask for the same laptop on the same dates and only the first approval works; a damaged return puts the asset under maintenance.

4. How we work and combine

Getting started (everyone, once)

Install **Node.js 20+** and **MySQL 8** (or XAMPP). Unzip the project, then:

```
cd server
npm install
copy .env.example .env      (then put your MySQL user and password in .env)
npm run db:setup            (creates the database with sample data)
npm run dev                 (server runs on port 5000)

cd client                   (in a second terminal)
npm install
npm run dev                 (open http://localhost:5173)
```

Sharing code with GitHub

#	Step
1	Shahan creates one GitHub repository, uploads the starter app and adds all four members.
2	Each member works on their own branch: `misbah`, `hooria`, `azlaan`, `hamza`.
3	When a module works, push your branch and open a Pull Request.
4	Shahan merges one Pull Request at a time and checks the app still runs.
5	Everyone pulls the latest `main` to see the others' finished work.

No GitHub? Zip only your own folders and send them to Shahan, who copies them into the master copy. It works because nobody edits the same files.

The plan, step by step

1. Setup
Everyone runs the starter app and logs in.

2. Server
Build your endpoints; test them in Postman.

3. Screens
Replace your placeholder pages.

4. Combine
Merge in order: Azlaan, Hamza, Hooria, Misbah.

5. Go live
Put it online and test the live link.

Final test after combining

Member registers → Searches "laptop" → Requests it for 3 days → Admin approves (member notified) → Admin issues → Returned as Damaged → Under Maintenance → Repaired → Dashboard updates → AI Assistant answers from real data → AI Recommendation shows only free assets

5. Going live

What	Where (free or cheap options)
Database	A hosted MySQL, for example Aiven (free plan) or Railway
App (server + website together)	Render or Railway. Build command `npm run build`, start command `npm start`
Secrets	Database login, JWT secret and the Claude API key go in the host's settings, never in GitHub

About the AI key: the AI features need a Claude API key, which is paid per use. Without a key the app still works: Recommendation uses simple keyword ranking and the Assistant shows a polite "unavailable" message, which is fine for practice.

Where to find more detail

File in the project	What's in it
`README.md`	How to install and run the app
`docs/API.md`	Every endpoint: what it takes and what it returns
`docs/modules/<your name>.md`	Your full technical task list with hints
`docs/SRS.pdf`	The requirements (look up any FR-xx ID here)
`server/src/config/constants.js`	Status names and limits (30 days, 50 AI uses, 2 MB images)